$$\nabla g\left(P_0\right) = \begin{bmatrix} \frac{\partial g}{\partial x}\left(P_0\right) \\ \frac{\partial g}{\partial y}\left(P_0\right) \end{bmatrix}$$

The geometrical interpretation of the gradient of a function is that it's a vector pointing in the direction of fastest growth of the function: that's why gradient descent actually uses the *negative gradient*, -∇g, which is simply the opposite of the gradient.

16.3.2 Geometrical Interpretation

Listing 16.4 gives a summarized description of the gradient descent algorithm.

It's important to understand that we don't have the full view of the cost function when we run gradient descent, and we don't "move" over the surface, rather at each step we only compute how much we should change the input variables, depending on the slope of the surface at that point: see figure 16.14 to get an idea of the steps (and figure 16.15 to get an idea of how it looks like with a 2D function's domain).

In our marble-race analogy, it's like if the track was swathed in a dense fog, and we could only see a feet away: enough to see where to aim for a cautious next step, because if we push the marble too hard, we risk sending it out of the track, or in the wrong direction.

Listing 16.4 Method gradientDescent

```
function gradientDescent(f, P0, alpha, maxSteps)
                  #1
```